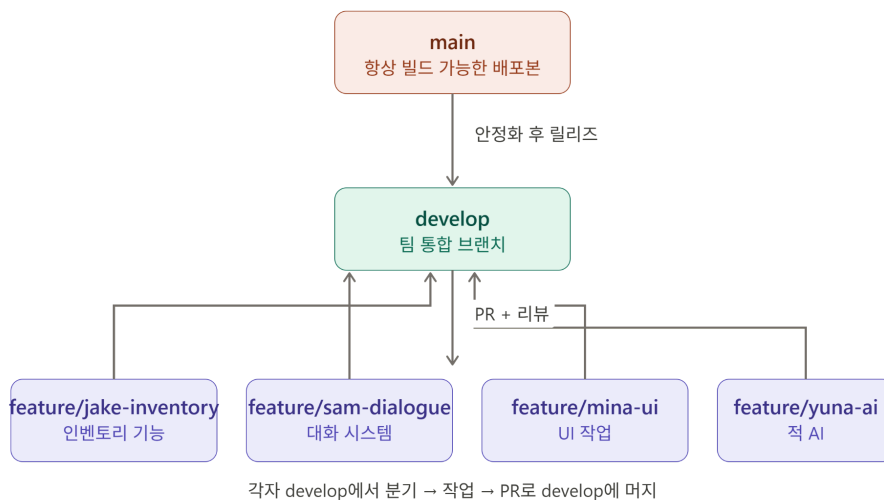


유니티 프로젝트 **Git** 브랜치 전략 가이드

1. 브랜치 구조

main → 항상 빌드 가능한 배포본
develop → 팀 통합 브랜치
feature/이름-기능 → 1인 작업 공간



main

- 항상 빌드 가능한 상태여야 하는 브랜치입니다.
- 여기에 직접 커밋하지 않습니다. 오직 **develop**을 통해서만 변경 사항이 들어옵니다.
- 한 마디로, 프로젝트의 "완성된 버전 보관소"입니다.

develop

- 팀원들의 작업이 모이는 곳입니다.
- 각자 작업을 마치면 **PR(Pull Request)**을 보내고, **develop**에 머지됩니다.
- 매일 작업하는 기준 브랜치입니다.

feature/이름-기능

- 한 사람이 한 가지 기능을 작업하는 개인 공간입니다.
- 이름 규칙: **feature/이름-기능** (예: **feature/jake-inventory**, **feature/sam-dialogue**)
 - 사람 이름만 쓰지 않고 기능까지 같이 적는 이유: 한 사람이 여러 기능을 작업할 수도 있어서, 브랜치 이름만 보고도 "이게 뭐였지?" 헛갈리지 않게 하기 위함입니다.
- 작업이 끝나서 **develop**에 머지되면, 이 브랜치는 삭제해도 됩니다. (지워도 기록은 **develop** 히스토리에 남습니다.)

작업 흐름 (요약)

1. **develop**에서 **feature/이름-기능** 브랜치를 새로 만든다.
2. 그 브랜치에서 작업한다.
3. 작업이 끝나면 **develop**으로 **PR**을 올린다.
4. (가능하면) 팀원 한 명이라도 리뷰를 보고 머지한다.
5. 머지가 끝난 **feature** 브랜치는 삭제한다.

커밋메시지 작성법

feat	feature, 새 기능	없던 기능이 생겼을 때	플레이어 대시 기능 추가
fix	버그 수정	잘못 동작하던 것을 고쳤을 때	무한 점프 버그 수정
chore	잡일	기능과 무관한 관리 작업 (버전 업데이트, 패키지 정리, 폴더 정리)	Unity 버전 업데이트
build	빌드 관련	빌드 설정, 플랫폼 설정 변경	Windows 빌드 설정 추가

알아두면 좋은 표준 태그도 있어요.

- **refactor**: 동작은 그대로이고 코드 구조만 정리 (함수 분리, 변수명 변경)
- **docs**: 문서 수정 (**README**, **SRS**, 주석)
- **style**: 들여쓰기, 공백 같은 코드 서식만 변경

2. 가장 자주 터지는 문제 — 씬(Scene) 충돌

유니티 프로젝트에서 **Git** 충돌이 제일 많이 나는 지점은 씬 파일(.unity) 입니다. 여러 사람이 같은 씬을 동시에 건드리면 **Git**이 자동으로 합쳐주지 못해서 충돌이 발생합니다.

해결 방법은 상황에 따라 세 가지가 있습니다.

방법 1. 씬 분리 (Additive Scene Loading)

씬 자체를 기능별로 쪼개서, 각자 자기 씬 파일만 작업하는 방식입니다.

MainScene (거의 안 바뀌는 베이스 씬)

- ├ Scene_Inventory ← Jake만 작업
- ├ Scene_Dialogue ← Sam만 작업
- └ Scene_UI ← Mina만 작업

메인 씬에서는 이 씬들을 동시에 겹쳐서 로드합니다 (순서대로 켜는 게 아니라 다 같이 띄우는 방식).

```
SceneManager.LoadSceneAsync("Scene_Inventory", LoadSceneMode.Additive);
SceneManager.LoadSceneAsync("Scene_Dialogue", LoadSceneMode.Additive);
```

각자 다른 .unity 파일을 건드리기 때문에 충돌이 거의 생기지 않습니다.

⚠ 주의: **Cesium**처럼 지형/3D 월드가 하나의 베이스 씬에 강하게 묶여 있는 경우, 씬을 쪼개기가 어렵습니다. 이런 경우에는 방법 2를 사용하세요.

방법 2. 프리팹 단위로 쪼개기 ★ 권장

씬은 한 명이 관리하고, 각자는 ****프리팹(Prefab)****을 만들어서 그 씬에 인스턴스만 가져다 놓는 방식입니다.

MainScene (Cesium 지형 + 프리팹들이 배치된 상태)

- ├ Inventory.prefab ← Jake 파일
- ├ DialogueUI.prefab ← Sam 파일
- └ MainUI.prefab ← Mina 파일

UI나 오브젝트를 프리팹으로 만들면, 각자 자기 프리팹 파일만 건드리게 되어 씬 파일 충돌이 줄어듭니다. **Cesium**처럼 지형이 이미 세팅된 프로젝트에서 가장 적합한 방식입니다.

방법 3. 어쩔 수 없이 같은 씬을 같이 써야 할 때

- 작업 시작 전, 채팅(디스코드/슬랙)에 한 마디 공유: "지금 **MainScene** 작업 중!"
 - 작업을 짧게 끊어서, 자주 커밋하고 자주 머지한다. (오래 들고 있을수록 충돌이 커집니다)
 - 충돌이 나면 유니티의 **Smart Merge(UnityYAMLMerge)** 도구로 직접 비교하며 해결한다.
-

3. 프리팹 작업 시 반드시 지킬 것

방법 2(프리팹 단위 분리)를 쓰더라도, 프리팹을 잘못된 방식으로 수정하면 썬 충돌이 그대로 다시 생깁니다.

문제 상황

썬에 이미 가져다 놓은 프리팹 인스턴스(오브젝트)를 썬 안에서 그냥 더블클릭 없이 수정하면 어떻게 될까요? 변경 내용은 프리팹 파일(.prefab)이 아니라 썬 파일(.unity)에 기록됩니다. 즉, 프리팹을 썼는데도 결국 썬 파일이 바뀌어 버려서, 프리팹 분리로 얻으려던 효과(썬 충돌 방지)가 사라집니다.

올바른 수정 방법 — 두 가지 중 하나를 반드시 따르기

방법 A. 프리팹 모드(Prefab Mode)에서 수정

- 프로젝트 창 또는 썬에 있는 프리팹을 더블클릭하면 회색 배경의 별도 화면(Prefab Mode)으로 들어갑니다.
- 이 화면에서 수정한 내용은 .prefab 파일에 바로 저장됩니다.
- 가장 안전하고 권장되는 방법입니다.

방법 B. 썬에서 수정 후 **Apply All**

- 썬에 배치된 인스턴스를 썬 안에서 직접 수정해야 하는 경우(예: 위치 조정처럼 인스턴스별로 달라야 하는 게 아니라 프리팹 자체를 바꿔야 하는 경우),
- 수정 후 인스펙터 상단의 **Overrides** → **Apply All**을 눌러서 변경 사항을 프리팹 파일에 완전히 적용해야 합니다.
- **Apply**를 누르지 않고 그대로 두면, 그 변경은 썬 파일에만 남아 있는 "오버라이드" 상태로 남고, 이는 곧 썬 충돌의 씨앗이 됩니다.

⚠️ 요약: 프리팹은 더블클릭해서 프리팹 모드로 들어가서 고치거나, 고친 다음 꼭 **Apply All**을 누르세요. 둘 다 안 하면 프리팹을 써도 썬 파일이 계속 바뀌어서 충돌이 그대로 발생합니다.

4. 체크리스트

작업 시작 전, 그리고 커밋 전 한 번씩 확인하세요.

- ☐ **main**에 직접 커밋하지 않았는가?
- ☐ **develop**에서 **feature/이름-기능** 브랜치를 새로 따고 작업했는가?
- ☐ UI/오브젝트는 가능하면 프리팹으로 만들어서 작업했는가?
- ☐ 프리팹을 고칠 때 더블클릭으로 **Prefab Mode**에 들어가서 수정했는가? (또는 수정 후 **Overrides** → **Apply All**을 눌렀는가?)
- ☐ 작업을 너무 오래 들고 있지 않고 짧게 끊어서 커밋했는가?
- ☐ PR 올리기 전, 본인 브랜치에서 **develop**의 최신 변경 사항을 받아왔는가? (`git pull origin develop`)